上海交通大学　数据库原理课程大作业（方案 1）

AuroraDB

自研关系数据库管理系统的设计、实现与测试报告

——兼论可信查询改写与智能优化扩展

组长　钟国鹏（SQL 编译与可信查询执行）

组员　曾子恒（数据存储与预测缓冲）

黄俊植（统计学习与性能优化）

路茗淳（AI 语义、交互与交付）

代码仓库：https://github.com/Guopeng-Zhong/AuroraDB

版本 1.2.0　•　2026 年 10 月

摘 要

本文报告数据库原理课程大作业 AuroraDB 的设计、实现与测试。AuroraDB 是一个仅依赖 Python 3.12 标准库、不调用任何现成数据库引擎的关系数据库管理系统，完整实现了课程要求的五个组件：以 4096 字节页、CRC 校验、稳定 RID 堆文件和 B+ 树组成的存储管理器；管理已提交页 LRU 缓存与事务暂存页的缓冲管理器；由词法分析、递归下降语法分析、名称绑定和访问路径选择构成的查询处理器；支持扫描、过滤、哈希连接、范围连接、聚合、排序与事务的物化执行引擎；以及浏览器工作台和命令行两种用户界面。事务采用 NO-STEAL 暂存与完整页镜像重做日志（WAL），可在提交中途崩溃后恢复。

在此内核之上，小组四人各负责一个研究扩展：（1）可信查询改写——候选改写必须依次通过“规则证据、全量结果核验、收益门槛”三道关卡，结论分为 proven / tested / unknown 三级，只有已证明且确实更快的改写才建议替换；（2）统计信息、轻量学习型代价模型与 3–6 表自适应连接规划，模型越出训练范围时回退启发式；（3）扫描感知的预测式缓冲淘汰与扫描准入控制；（4）带批处理、去重、精确缓存和预算约束的 AI 语义关系算子。

系统共有 279 项自动化测试全部通过，其中包含 180 条与 SQLite 对照的随机差分查询；各项实验以固定种子生成，原始数据以 JSON 形式随代码提交。在公开数据集 Chinook 上，可信改写把 `GenreId = 2 OR GenreId = 6` 改写为 `IN (2, 6)`，扫描行数由 3 503 降为 211、文件读页由 158 降为 28；自适应连接规划把三表连接的候选比较次数由 20 000 降为 200。

关键词：关系数据库；B+ 树；预写日志；查询优化；查询改写验证；缓冲管理；语义算子

Abstract

This report presents AuroraDB, a relational database management system built for a database course using only the Python 3.12 standard library. It implements a page-based storage manager with CRC-checked 4 KB pages, heap files with stable record identifiers and B+ tree indexes; an LRU buffer manager with transaction-private staged pages; a query processor with a hand-written lexer, recursive-descent parser, name binding and access-path selection; a materializing execution engine; and both a web workbench and a command-line interface. Durability is provided by NO-STEAL staging and a full-page-image redo log. On top of this kernel, four extensions are explored: trusted query rewriting that separates rule-based proof from data-level testing and gates replacement on measured benefit; lightweight learned cost models with adaptive join ordering and out-of-distribution fallback; scan-aware predictive buffer eviction; and budgeted LLM-backed semantic operators. All 279 automated tests pass, including 180 randomized differential queries checked against SQLite.

Keywords: relational database; B+ tree; write-ahead logging; query optimization; query rewrite verification; buffer management; semantic operators

目录

摘要 **I**

Abstract **II**

1 引言 **1**

1.1 课题背景与要求 1

1.2 主要工作 1

1.3 报告结构 1

2 相关工作 **2**

2.1 DBMS 体系结构与存储 2

2.2 查询优化 2

2.3 查询改写的等价验证 2

2.4 缓冲管理与语义算子 2

2.5 本项目的借鉴边界 2

3 需求分析与总体设计 **3**

3.1 需求分析 3

3.2 总体架构 4

3.3 模块划分与源文件 4

3.4 一条 SQL 的处理流程 4

4 存储管理器 **5**

4.1 文件组织 5

4.2 页格式 5

4.3 堆文件与稳定 RID 6

4.4 B+ 树索引 6

4.5 预写日志与崩溃恢复 6

4.6 系统目录与一致性检查 7

5 缓冲管理器 **7**

5.1 已提交页缓存与事务暂存 7

5.2 扫描感知的预测式淘汰 8

6 查询处理器 **8**

6.1 词法分析 8

6.2 递归下降语法分析 9

6.3 名称绑定与类型检查 9
6.4 访问路径选择 9
6.5 连接规划 9
6.6 统计信息与学习型代价模型 10

7 执行引擎 10
7.1 算子与物化执行 10
7.2 NULL 语义与类型规则 10
7.3 事务 11
7.4 EXPLAIN 与执行计数 12

8 研究扩展与创新点 12
8.1 可信查询改写（钟国鹏） 12
8.2 统计学习与自适应连接规划（黄俊植） 15
8.3 扫描感知的预测缓冲（曾子恒） 15
8.4 受预算约束的 AI 语义算子（路茗淳） 15

9 用户界面与使用说明 16
9.1 浏览器工作台 16
9.2 命令行界面 18
9.3 运行方式 18
9.4 安全设计 18

10 测试与实验 18
10.1 测试方法 18
10.2 测试结果 19
10.3 实验设置 19
10.4 可信改写 19
10.5 学习型优化 20
10.6 预测缓冲 21
10.7 语义算子的批处理与缓存 22
10.8 公开数据集 22
10.9 有效性威胁 22

11 小组分工与完成情况 22

12 局限与展望 23

13 结论 24

参考文献 **25**

A 复现步骤与仓库结构 **26**

1 引言

1.1 课题背景与要求

数据库管理系统（DBMS）把数据的物理存储、并发访问、故障恢复与声明式查询封装在一起，是信息系统的基础软件 [1, 2]。课程大作业方案 1 要求小组按照软件工程规范设计并实现一个数据库管理系统，至少包含存储管理器、缓冲管理器、查询处理器、执行引擎和用户界面五个组件，并提交源代码、可执行程序、使用说明、各模块实现方法与测试报告，以及小组分工与完成情况说明。

本组实现的系统命名为 AuroraDB。我们给自己设定了三条原则：

1. **全部自研**：运行时只使用 Python 标准库（`pyproject.toml` 中 `dependencies = []`），查询、堆文件、B+ 树、页缓存和日志均由本项目实现；SQLite 只在测试中作为对照结果的来源。
2. **可观察**：每条查询都能看到执行计划、实际扫描行数、读页次数、缓冲命中等真实计数，便于教学演示，也便于做可复现的实验。
3. **正确性优先**：任何优化（索引选择、改写、缓存策略、AI 算子）在证据不足时都回退到确定、保守的路径，不用错误结果换取速度。

1.2 主要工作

1. 实现了一个完整的单机关系数据库内核（约 6 500 行 Python 源码），支持 DDL、DML、内连接 / 左连接 / 交叉连接、分组聚合、排序分页、显式事务、`EXPLAIN ANALYZE` 和一致性检查。
2. 提出并实现了“可信查询改写”闭环：把“规则推导出的等价”与“当前数据上测出来一致”严格区分，并以实测收益门槛决定是否建议替换（第 8.1 节）。
3. 实现了统计信息采集、轻量学习型代价模型、3–6 表连接顺序动态规划、扫描感知的预测缓冲，以及受预算约束的 AI 语义算子，每项扩展都保留默认的启发式 / LRU 路径作为对照与回退。
4. 构建了 279 项自动化测试和 5 组可复现实验，导入公开数据集 Chinook（11 张表、15 607 行）用于更真实的演示。

1.3 报告结构

第 2 节介绍相关工作；第 3 节给出需求分析与总体设计；第 4–7 节依次介绍存储管理器、缓冲管理器、查询处理器和执行引擎；第 8 节介绍四项创新扩展；第 9 节是用户界面与使用说明；第 10 节是测试与实验；第 11 节说明小组分工；最后讨论局限并总结全文。

2 相关工作

2.1 DBMS 体系结构与存储

关系模型由 Codd 提出 [1]，经典教材把 DBMS 划分为查询处理、存储管理和事务管理等层次 [2, 3]，本项目的组件划分与之一致。B 树及其变体 B+ 树 [4, 5] 是磁盘上有序索引的事实标准：数据只在叶节点、叶节点之间用链表相连，因而等值查找和范围扫描都只需一次自顶向下的下降。恢复方面，ARIES [8] 以 WAL、STEAL / NO-FORCE 和三阶段恢复成为工业标准；本项目在单写者、单事务的约束下采用更简单的 NO-STEAL + 完整页镜像重做，正确性论证更直接。

2.2 查询优化

System R 开创了基于代价的访问路径选择与连接顺序动态规划 [6]；Graefe 系统总结了迭代器 / 物化执行、哈希连接与排序等查询执行技术 [7]。Leis 等人的实验表明，真实负载下基数估计误差往往是导致坏计划的主因 [10]。近年的学习型基数估计希望用模型降低误差：PRICE 通过跨数据库预训练降低训练准备成本 [14]，CardOOD 关注训练与测试负载分布不一致（Out-of-Distribution）时的鲁棒性 [15]。

2.3 查询改写的等价验证

查询改写在保持语义的前提下改变 SQL 写法以获得更好的计划。判定两条一般 SQL 是否等价是不可判定问题，因此 Cosette 等工具只能对受限片段给出自动证明 [11]。随着大语言模型（LLM）被用于改写 SQL，如何确认 LLM 输出的改写没有改变语义成为落地的关键障碍。Narasayya 与 Chaudhuri [12] 提出利用查询优化器内部本来就可靠的变换规则来验证改写的等价性，验证不了就如实报告；ReSequel [13] 则用元数据模板约束 LLM 生成候选，在抽样数据上执行校验，并按实测性能排序。

2.4 缓冲管理与语义算子

LRU 是最常用的缓冲替换策略，LRU-K 用最近 K 次访问时间改善对扫描污染的抵抗力 [9]。Khalaji 等人利用扫描进度预测页面的下一次访问时间，并只在少量随机样本中挑选淘汰页以降低开销 [16]。iPDB 把 LLM / ML 调用作为关系算子嵌入 SQL，并做输入去重、多行打包等语义查询优化 [17]。

2.5 本项目的借鉴边界

表 1 总结了本项目从上述工作借鉴的思想与缩减实现的边界。我们只借鉴方法思路，不复现原论文的完整系统，也不把论文中的性能数字当作本项目的结论。

表 1　相关研究与本项目的缩减实现

文献	原研究重点	本项目的实现
Narasayya 等 [12]	用优化器的变换规则验证 LLM 改写的等价性	少量手写可信规则及其前置条件；规则推不出的候选最多标为 tested
ReSequel [13]	模板化生成、抽样校验、按实测性能排序	候选必须实测；把验证成本计入是否替换的判断
PRICE [14]	跨数据库预训练基数估计	借鉴统计特征设计，实现标准库 ridge 代价模型，不加载预训练模型
CardOOD [15]	训练 / 测试分布偏移下的鲁棒估计	用训练集外的查询范围做留出评估，越界时回退启发式
预测缓冲 [16]	扫描进度预测 + 小样本淘汰	串行扫描感知、热点保护与扫描准入，证据不足时回退 LRU
iPDB [17]	SQL 内的 LLM / ML 语义算子与优化	显式的筛选、分类、聚合管线，含去重、批处理、缓存与预算

3　需求分析与总体设计

3.1　需求分析

功能需求　对应课程要求的五个组件：（1）存储管理器在磁盘上按关系模型存储数据，并提供索引等高效访问方式；（2）缓冲管理器管理内存缓冲区，负责与磁盘之间的页 I/O；（3）查询处理器编译 SQL、生成并优化查询计划；（4）执行引擎执行查询计划；（5）用户界面接受 SQL、显示结果，提供图形界面与命令行。此外，系统需要支持事务的提交、回滚与崩溃恢复。

非功能需求

- **正确性**：结果遵循 SQL 的 NULL 三值逻辑和类型规则；与 SQLite 的差分测试一致。
- **持久性与可恢复**：已确认提交的事务在进程崩溃后不丢失，未提交的修改不落盘。
- **可观察与可复现**：执行计划、读页次数、缓冲命中等计数真实可见；实验固定随机种子，原始数据随代码交付。
- **安全**：SQL 只被解析为数据结构，从不交给 `eval`；工作台只监听本机回环地址；AI 密钥只放在本地配置文件中，不进入仓库和安装包。
- **可移植**：Windows 提供单文件可执行程序 `dist/AuroraDB.exe`；macOS / Linux 可直接用源码运行。

3.2 总体架构

图 1 给出 AuroraDB 的组件结构。一条 SQL 从用户界面进入查询处理器，被编译成执行计划；执行引擎按计划向缓冲管理器请求页；缓冲区未命中时，由存储管理器从磁盘文件读取并校验。研究扩展以旁路模块的形式挂接在对应组件上，默认关闭或保持启发式行为。

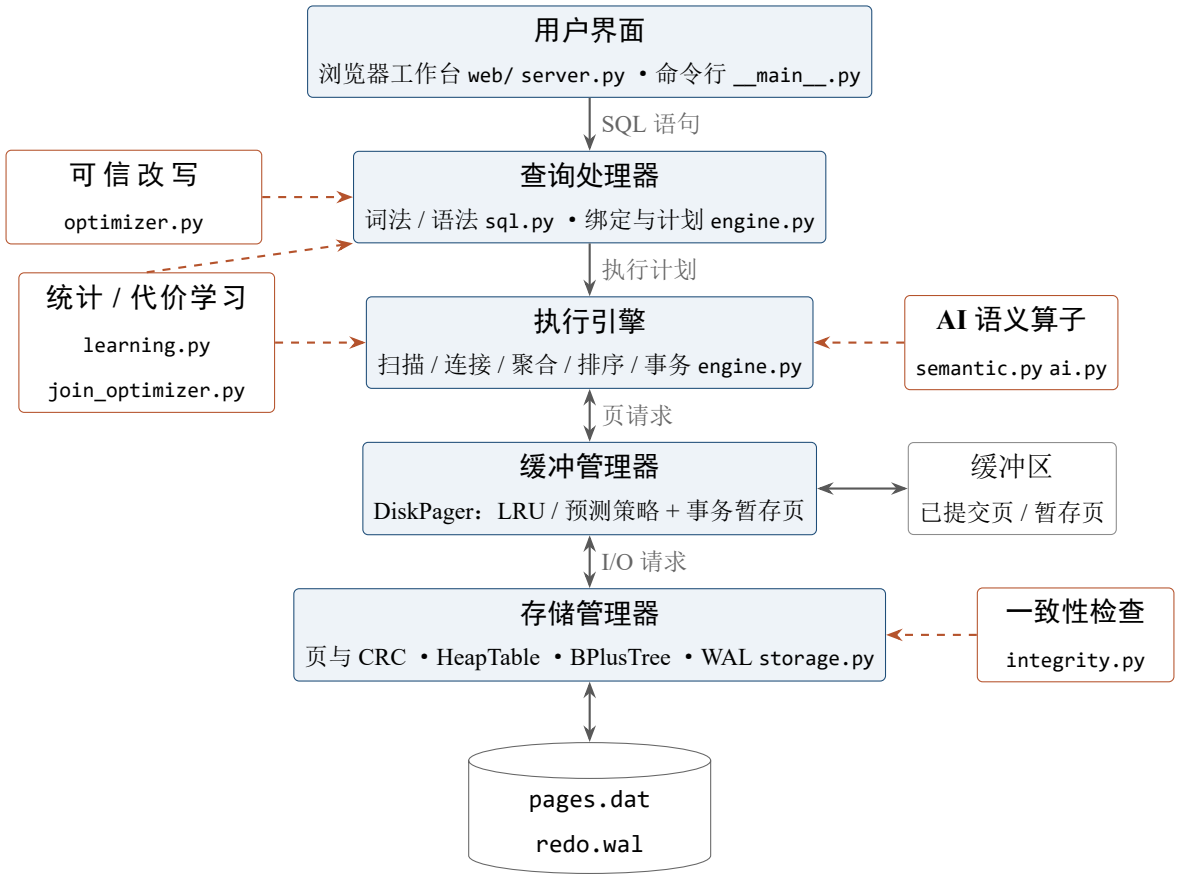


图 1 AuroraDB 组件结构（蓝色为课程要求的五个组件，橙色虚线为研究扩展模块）

3.3 模块划分与源文件

表 2 列出各组件对应的源文件、代码规模和负责人。

3.4 一条 SQL 的处理流程

图 2 以演示中使用的查询为例，说明 SQL 文本经过的各个阶段。每个阶段的输出都是普通的数据结构，可以在工作台和测试中直接检查。

表 2　组件、源文件与负责人

组件	主要源文件（auroradb/ 下）	行数	负责人
存储管理器	storage.py（页、堆表、B+ 树、WAL、DiskPager）、integrity.py	1 169 + 473	曾子恒
缓冲管理器	storage.py 中的 DiskPager 与预测策略	（同上）	曾子恒
查询处理器	sql.py（词法、语法）、engine.py（绑定、访问路径）	684 + 1 250	钟国鹏
统计与连接规划	learning.py、workload.py、join_planning.py、join_optimizer.py	1 048	黄俊植
执行引擎与事务	engine.py（算子、事务、EXPLAIN）	（同上）	钟国鹏
可信改写	optimizer.py、optimizer_demo.py	487	钟国鹏
AI 语义算子	semantic.py、ai.py	758	路茗淳
用户界面与交付	server.py、__main__.py、web/、tools/	621 + 前端	路茗淳

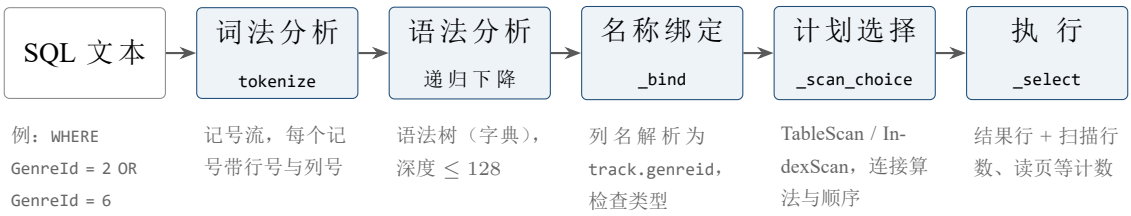


图 2　一条 SQL 在 AuroraDB 中的处理流程

4　存储管理器

4.1　文件组织

每个数据库是一个目录，包含三个内部文件：pages.dat 为固定大小页文件，第 p 页位于字节偏移 $p \times 4096$；redo.wal 为单个事务的完整页镜像重做日志，正常提交后清空；writer.lock 是操作系统独占锁的载体（Windows 使用 msvcrt.locking，POSIX 使用 flock），保证一个目录同时只有一个写者连接。

4.2　页格式

所有页大小为 4096 字节，头部 20 字节以小端编码（struct '<8sIII'），其后为 UTF-8 JSON 页体（图 3）。CRC32 的输入为页号、有效长度以及全部 4076 字节页体，因而内容、长度、页号和尾部填充的损坏都能被发现；读到损坏页时返回 StorageError，不会自动覆盖。CRC 用于检测非恶意的数据损坏，不是密码学签名。

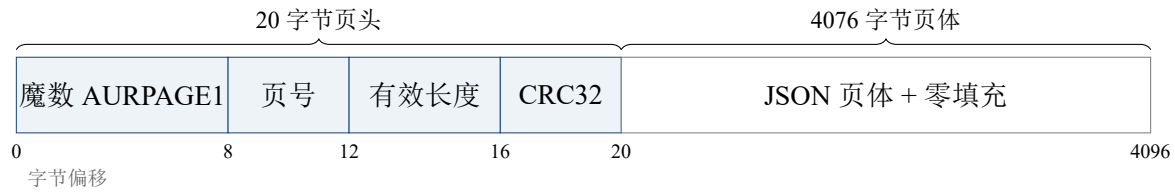


图 3　4096 字节页格式

页体是一个 JSON 对象，用 `kind` 字段区分页类型：系统目录（`catalog_root` / `catalog_chunk`）、堆页（`heap`）、溢出页（`overflow`）、B+ 树叶节点和内部节点。JSON 使用紧凑分隔符和确定的键顺序，拒绝 NaN 与 Infinity。选用 JSON 而不是二进制行格式，是为了让页内容在调试与一致性检查时可以直接阅读，代价是空间利用率较低。

4.3　堆文件与稳定 RID

堆页形如 `{"kind":"heap","rows":[{"v":{...}},null,...]}`，记录标识 RID 为 `[页号, 槽号]`，可以跨重启保存，供 B+ 树回表。删除只把槽置为 `null`，不移动其他记录，所以其他 RID 保持不变；后续插入可复用空槽。更新时若原页空间不足，则把新内容写入溢出页，并把原槽改为指向溢出页的引用，从而保持 RID 与索引引用不变。表元数据中的 `insert_hint` 与 `reusable_pages` 记录当前追加页和有空槽的页，避免每次插入都遍历全部堆页。

4.4　B+ 树索引

叶节点保存 `[[key, RID], ...]` 和指向右兄弟的 `next`，内部节点保存分隔键与子页号，子节点数恒为键数加一。为支持非唯一索引，树中的排序键是复合键 (key, RID)：同一个键值的不同记录可以跨越多个叶节点，等值查找从 $(key, [-1, -1])$ 定位最左叶，再顺着 `next` 链读出全部重复键；范围扫描同样只需一次下降。叶节点按编码字节数平衡分裂并维护叶链，根分裂时立即更新目录中的根页号，使新根与数据在同一事务中持久化。NULL 不进入索引（唯一约束因此允许多个 NULL）；单个键的编码不超过 1024 字节，保证节点总能合法分裂。删除采用延迟合并策略，只移除叶条目，保留现有分隔键与空叶，`validate()` 检查顺序、分隔范围、等深与叶链完整性。

4.5　预写日志与崩溃恢复

事务修改先写入缓冲管理器中的暂存页（第 5 节），提交时按图 4 的顺序持久化。WAL 由 BEGIN（B）、页镜像（P）和带 SHA-256 摘要的 COMMIT（C）三类记录组成，每帧有长度和 CRC32；含 N 个不同页的事务，日志大小为 $90 + 4113N$ 字节。

恢复的不变量如下：第③步之前崩溃，数据文件未被修改，未完成的日志可以丢弃；第③步之后、第⑤步之前崩溃，重启时重新写入全部页镜像，重复重做是幂等的；第⑤步之后清理日志失败也不会报告虚假的回滚。只有完整通过帧 CRC、页 CRC、页

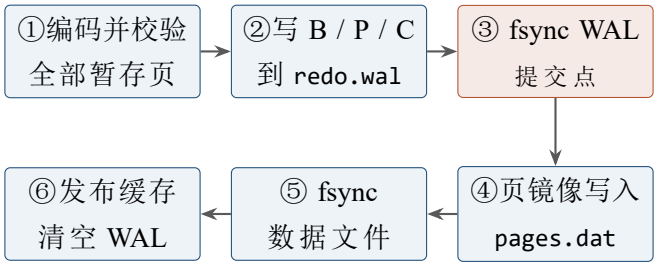


图 4　提交流程：第③步之后日志已足以重做整个事务

号范围、数量一致性和事务 SHA-256 校验的日志才会被重做；遇到无法确认的损坏时，系统保留原始文件并报错，而不是把损坏记录悄悄当作成功或失败。提交过程中发生异常时，pager 被标记为 *poisoned*，拒绝继续读写，必须重新打开才能确定持久化状态。该方案是物理页镜像重做，而不是完整的 ARIES 协议 [8]；它的正确性依赖操作系统 `fsync` 按约定工作。

4.6　系统目录与一致性检查

物理第 0 页保存目录清单，逻辑目录（全部表、列、索引和堆页列表）按 2900 字节切片后分别存入多个目录页，因此表数量不受单页容量限制；目录、堆页和索引页的修改处于同一个页镜像事务中。

`integrity.py` 提供只读的一致性检查（SQL 语句 `CHECK DATABASE` 或工作台“一致性检查”面板），逐页校验 CRC 与页类型、检查堆记录与列类型、B+ 树的有序性与叶链，并核对每个索引条目与堆记录一一对应。图 5 是对 Chinook 数据库的检查结果：632 页、11 张表、22 个索引、15 607 条记录，没有发现问题。



图 5　工作台“一致性检查”面板（Chinook 数据库）

5　缓冲管理器

5.1　已提交页缓存与事务暂存

缓冲管理器（`DiskPager`）区分两类内存页：

- **已提交页缓存：**由 `OrderedDict` 实现的有界 LRU，默认 64 页（256 KiB）。命中时移到最近使用端，超过容量时淘汰最久未使用的页。读出和写回都复制页对象，避

免调用者直接修改缓存中的字典而绕过写路径。

- **事务暂存页**：事务中修改过的页保存在独立的 `_staged` 字典中。读页时优先返回暂存版本，实现“读到自己的写”；暂存页永远不会因为 LRU 淘汰而提前写入数据文件（NO-STEAL）。回滚时直接丢弃暂存页并恢复新页分配边界。

工作台顶部实时显示缓冲区命中率、已用缓存页数、实际读页与写页次数（见图 12），它们统计的是应用层页读写，不等同于操作系统的设备 I/O。

5.2 扫描感知的预测式淘汰

LRU 的典型弱点是“扫描污染”：一次大范围扫描会把频繁访问的热点页全部挤出缓存。借鉴文献 [16]，曾子恒实现了可切换的预测策略（`set_buffer_policy('predictive', sample_size=8)`）：

1. 堆表和 B+ 树扫描向缓冲管理器登记扫描上下文，并提供“下一页”提示；扫描结束时在 `finally` 中注销。
2. 需要淘汰时，只从少量驻留页样本中挑选候选（样本数受 `sample_size` 与缓存大小限制），综合受限的访问频率、近期性和扫描信息，优先淘汰已被扫描越过、短期内不会再用的页，保护热点页。
3. **扫描准入**：缓存已满、新读入的页只属于扫描、驻留页有更强的点访问历史、且没有活动扫描需要该页时，新页直接交给调用者而不进入缓存，避免污染。
4. 证据不足时回退到 LRU。默认策略仍是 LRU，可随时切换回来。

策略只作用于已提交页，不影响事务暂存与 WAL 流程；预测淘汰次数、采样次数、回退次数和准入拒绝次数都单独计数。第 10.6 节给出与同版 LRU 的对照实验。

6 查询处理器

查询处理器负责把 SQL 文本编译为可执行的计划。支持的 SQL 子集包括：CREATE / DROP TABLE 与 INDEX（含 UNIQUE）；INTEGER、REAL、TEXT、BOOLEAN 四种类型及常用别名；单列 PRIMARY KEY、UNIQUE、NOT NULL 与常量 DEFAULT；多行 INSERT、UPDATE、DELETE；SELECT、DISTINCT、WHERE、INNER / LEFT / CROSS JOIN 与逗号连接、GROUP BY、HAVING、ORDER BY、LIMIT / OFFSET；算术、比较、AND / OR / NOT、IN、BETWEEN、IS NULL、LIKE；COUNT、SUM、AVG、MIN、MAX 等聚合与常用标量函数；BEGIN / COMMIT / ROLLBACK、CHECKPOINT、CHECK DATABASE、EXPLAIN 与 EXPLAIN ANALYZE。子查询、窗口函数、RIGHT / FULL JOIN、外键、视图与触发器不在支持范围内，遇到时明确报错。

6.1 词法分析

`sql.py` 中的 `tokenize` 把 SQL 文本切分为关键字、标识符、数字、字符串和运算符记号，每个记号记录行号与列号，因此任何报错都能指出具体位置。词法层处理分

号分隔的多语句脚本、转义引号、双引号或反引号标识符、行注释和可嵌套的块注释；标识符统一转为小写。

6.2 递归下降语法分析

语法分析器 `_Parser` 为每个运算优先级编写一个函数，优先级从低到高依次为 OR、AND、NOT、比较（含 IN、BETWEEN、LIKE、IS NULL）、加减、乘除取模、一元运算和基本表达式。例如 `a = 1 OR b = 2 AND c = 3` 会被解析为 `a = 1 OR (b = 2 AND c = 3)`。输出的语法树由普通的字典和列表组成，例如一个 OR 节点是 `kind` 为 `binary`、`op` 为 `OR`、带 `left` 与 `right` 两个子节点的字典；语法树只是数据，不会被当作代码执行。语法树深度限制为 128 层，防止恶意的深层括号导致 Python 递归溢出，超限时返回普通的语法错误。

6.3 名称绑定与类型检查

`_bind` 把语法树中的列名解析为“表别名. 列名”的完全限定形式，检查列是否存在、是否有歧义，推断表达式类型并在执行前拒绝不兼容的比较；GROUP BY 查询会严格检查非聚合列。绑定失败时给出带位置的中文错误信息，执行器不会接触到未绑定的名称。

6.4 访问路径选择

对每个基表，`_scan_choice` 决定使用全表扫描（TableScan）还是索引扫描（IndexScan）：

1. 把 WHERE 按顶层 AND 拆成若干合取项。
2. 只有两种形状的合取项可以使用索引：**列比较符常量**（`= < <= > >=`）和 **列 IN (常量, …)**。
3. 对每个可用索引的合取项，在 B+ 树中查出候选 RID，选择候选最少的那个。
4. 启发式模式下，候选数不超过全表的 35%（`len(rids) <= max(1, total * .35)`）才走索引，否则离散的回表读取不如一次顺序扫描划算；自适应模式下由代价模型比较两条路径（第 6.6 节）。
5. 无论选择哪条路径，读出的行都会再用完整的 WHERE 过滤一遍，保证结果正确。

这一设计对 AND 天然正确：任何一个合取项查出的候选都包含最终答案。但对 OR 不成立——只查 OR 的一边会漏掉满足另一边的行，所以规划器不处理 OR 节点，`GenreId = 2 OR GenreId = 6` 只能全表扫描。这正是第 8.1 节可信改写的出发点。

6.5 连接规划

连接处理由黄俊植负责，分为规范化与顺序搜索两部分：

- **规范化**（`join_planning.py`）：名称绑定后，把纯内连接中逗号写法、`CROSS JOIN`

`+ WHERE` 和 `JOIN ... ON` 统一为同一种表示；跨表的 WHERE 条件放到最早可计算的连接前缀，单表条件转为输入过滤。存在 LEFT JOIN 时不跨越外连接屏障改写，可空侧的 WHERE 条件仍在连接后检查。

- **物理算法**：等值条件使用哈希连接（HashJoin）；直接列之间的 `< <= > >=` 条件使用 RangeJoin——对右输入排序，对每个左行二分定位候选区间，再检查完整 ON 条件，复杂度约为 $O(R \log R + L \log R + K)$；其余条件保留嵌套循环。
- **顺序搜索**（`join_optimizer.py`）：自适应模式下，对 3–6 张表的连通等值内连接做左深计划的动态规划，用统计信息估计各前缀的中间结果大小；模型不可用或查询超出支持范围时保留原 SQL 的连接顺序。

6.6 统计信息与学习型代价模型

`Database.analyze_statistics()` 以只读方式采集每列的行数、NULL 数、不同值数量（NDV）、取值范围和直方图。在此基础上，`learning.py` 为 TableScan 与 IndexScan 各训练一个标准库实现的 ridge 回归读路径代价模型：至少有 8 个冷缓存样本且有 3 种不同的特征向量时模型才就绪。自适应模式（`set_optimizer_mode('adaptive')`）在训练特征范围内使用模型选择路径，越出范围时回退启发式；过滤基数的校准施加单调约束。写入、统计更新或缓冲策略切换都会使模型与结论失效。默认的 heuristic 模式保持原有的规则选择，便于对照。

7 执行引擎

7.1 算子与物化执行

执行引擎把计划组织为一棵算子树，表 3 列出主要算子。执行采用**物化**方式 [7]：每个算子把结果全部算完后再整体交给上层。这种方式实现简单、每一步处理的行数容易统计，代价是中间结果较大时占用内存，因此执行器对扫描行数、连接比较次数和中间结果规模设置了协作式预算。

图 6 是 Chinook 上“曲目最多的前 10 位艺术家”查询的执行计划。读法是缩进越深越先执行：三张表各扫描一次（275 + 347 + 3 503 = 4 125 行，即工作台显示的扫描行数），两次哈希连接把曲目与专辑、艺术家拼接，再按艺术家分组计数、排序并取前 10 行。图中的行数是规划器在执行前的估计值，实际值以 EXPLAIN ANALYZE 为准。

7.2 NULL 语义与类型规则

表达式求值遵循 SQL 的三值逻辑：比较中任一侧为 NULL 时结果为“未知”，`sql_and` / `sql_or` 按真值表合成，WHERE 只保留结果为“真”的行。除法返回实数、除以零返回 NULL；DISTINCT 与 GROUP BY 使用带类型的键，BOOLEAN 不与数值 0 / 1 合并；连接在选择物理算法之前先校验比较类型，避免哈希连接与嵌套循环在类

表 3　执行引擎的主要算子

算子	作用
`TableScan`	按堆页顺序读出整张表
`IndexScan`	按 B+ 树给出的 RID 回表读取
`Filter`	按 WHERE 条件过滤（可下推到基表扫描）
`HashJoin`	等值连接：对一侧输入按连接键建哈希表，另一侧逐行探测
`RangeJoin`	范围连接：排序 + 二分定位候选区间
`NestedLoopJoin`	其他连接条件的通用实现
`HashAggregate`	GROUP BY 分组与 COUNT / SUM / AVG / MIN / MAX
`Project / Distinct`	计算输出列；按带类型的键去重
`Sort / Limit`	排序（NULL 升序在前、降序在后）；分页

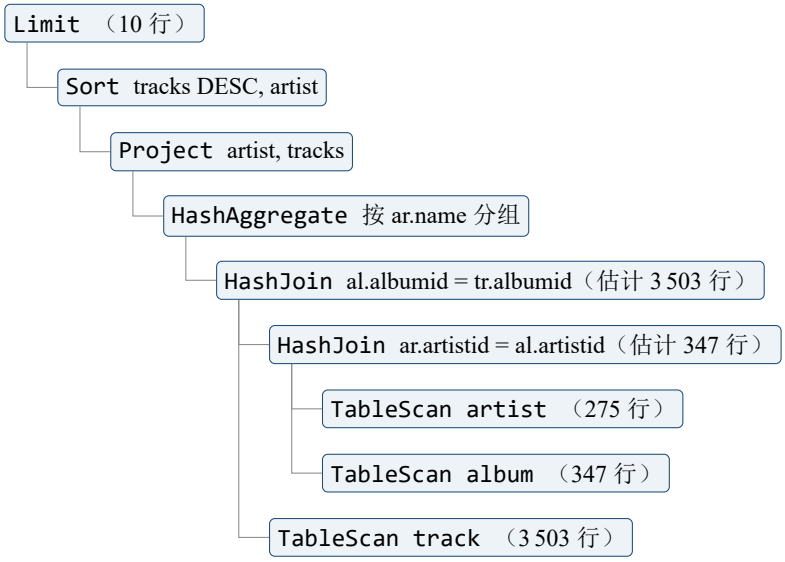


图 6　Chinook 三表连接聚合查询的执行计划树

型不兼容时给出不同结果。

7.3　事务

默认自动提交，每条语句是一个事务。`BEGIN` 之后的修改只进入暂存页，其他查询看不到；`COMMIT` 按第 4 节的流程持久化；`ROLLBACK` 丢弃暂存页。显式事务中任何一条语句出错，整个事务回滚，不会出现部分生效。系统是单写者设计，不提供并发事务与 MVCC。

7.4 EXPLAIN 与执行计数

`EXPLAIN` 只生成计划不读数据；`EXPLAIN ANALYZE` 真正执行查询，并在计划中附上实际行数、扫描行数、索引查找次数、连接比较次数和耗时。工作台的“执行计划”页与“扫描对比”工具都基于这些真实计数。

8 研究扩展与创新点

8.1 可信查询改写（钟国鹏）

8.1.1 问题

查询改写是“换一种写法，意思不变，执行更快”。在第 6.4 节的规划器下，下面两条查询语义相同，但执行代价差别很大：

```
-- original: TableScan, 3503 rows
SELECT TrackId, Name, Milliseconds FROM Track WHERE GenreId = 2 OR GenreId = 6;
-- rewritten: IndexScan, 211 rows
SELECT TrackId, Name, Milliseconds FROM Track WHERE GenreId IN (2, 6);
```

改写的风险在于“改错了，结果就错了”。一般 SQL 的等价性不可判定 [11]，而“在当前数据上结果相同”只是等价的必要条件：例如候选 `GenreId IN (6, 2) AND Milliseconds >= 0` 在现有数据上与原查询结果完全一样，但只要插入一条时长为负数或为 NULL 的错误数据，候选就会少返回一行。由 LLM 生成的改写同样没有等价保证 [12, 13]。因此我们需要一个机制，**只接受能证明没改错、全量结果一致且确实更快的改写，其余一律保留原查询**。

8.1.2 可信规则

`optimizer.py` 中的 `trusted_rewrite` 只对 WHERE 子句应用三条手写规则，投影、分组、排序和连接结构不变：

- `or-equalities-to-in`：同一列上 2–32 个“列 = 常量”用 OR 连接时合并为 IN。前置条件为：每一项都是等值比较、一侧是同一个列、另一侧是常量，且常量类型与列类型匹配（避免隐式类型转换带来的差异）；任一条件不满足即原样返回。
- `duplicate-and`：删除 AND 中完全相同的合取项。
- `where-true`：删除恒为真的条件。

以 OR → IN 为例，WHERE 对每一行独立求值，一行的 GenreId 只有表 4 中四种情况，两种写法的真值在每种情况下都相同，因此在任何数据上都选出完全相同的行（WHERE 只决定行的去留，不改变重复次数）。

表 4　OR → IN 改写的逐行真值对照（含 NULL）

该行 GenreId	`GenreId = 2 OR GenreId = 6`	`GenreId IN (2, 6)`	该行
2	真 OR 假 = 真	真	两边都保留
6	假 OR 真 = 真	真	两边都保留
其他值	假 OR 假 = 假	假	两边都丢弃
NULL	未知 OR 未知 = 未知	未知	两边都丢弃

8.1.3　三道关卡

`optimize(db, sql, candidate_sql=None)` 实现图 7 的闭环。候选可以由本地规则生成，也可以由用户手写或由大模型给出，系统对所有来源一视同仁地检查。实验在执行前还会拒绝带 LIMIT / OFFSET 的候选以及 ORDER BY 不一致的候选。

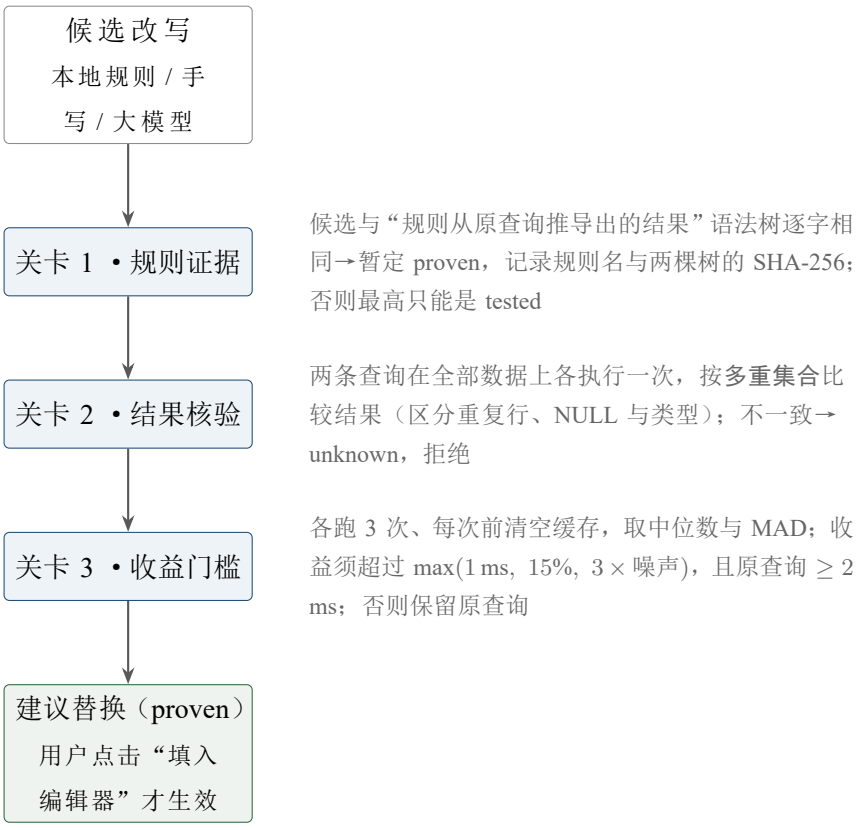


图 7　可信改写的三道关卡

收益门槛的完整定义为（`optimizer.py`）：

$$\text{margin} = \max\big(1.0,\ 0.15\,t_{\text{orig}},\ 3(\text{MAD}_{\text{orig}} + \text{MAD}_{\text{cand}})\big), \tag{1}$$

$$\text{replace} = (t_{\text{orig}} \geq 2.0)\ \wedge\ (t_{\text{orig}} - t_{\text{cand}} \geq \text{margin}), \tag{2}$$

其中 t 为 3 次冷缓存执行耗时的中位数（毫秒）。原查询本身很快时，验证的开销超过可能的收益，系统选择不替换，这一点借鉴了 ReSequel 把验证成本计入收益的思想

[13]。结论分为三级：**proven**（规则可推导且结果一致）、**tested**（只是当前数据上一致，不建议自动替换）、**unknown**（结果不同或无法比较）。全过程只执行 SELECT，不修改任何数据。

“已证明”的含义是：候选由经过审查的规则推导而来，推导过程可以重放检查。它不是通用的等价证明器——那在理论上不存在。这一“证明不了就如实说未知”的分级思想借鉴自文献 [12]。

8.1.4 演示结果

图 8 是在 Chinook 上对上述 OR 查询运行可信改写的结果：三道关卡全部通过，扫描行数由 3 503 降为 211，文件读页由 158 降为 28，查询耗时由 75.4 ms 降为 37.4 ms（2.02 倍），收益 37.99 ms 超过门槛 16.05 ms（本次测量中三倍噪声项大于 15% 项），因此建议替换。各次运行的毫秒数会随机器负载波动，但扫描行数与读页数是确定的。作为对照，图 9 的“扫描对比”工具显示，原 OR 查询在“强制全表扫描”和“优化器选择”两条路径下都读了 3 503 行——规划器确实无法为 OR 使用索引。

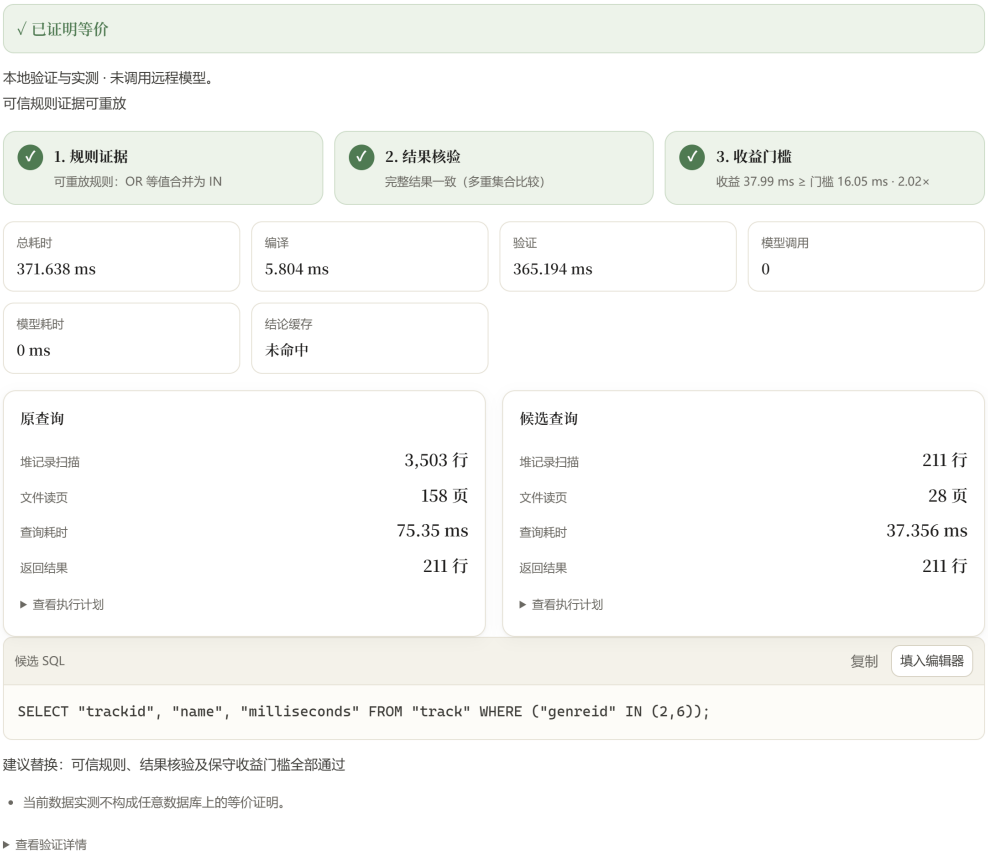


图 8　工作台“研究实验→可信改写”面板：OR → IN 改写通过三道关卡



图 9 “扫描对比”工具：原 OR 查询的两条访问路径都只能全表扫描

8.2 统计学习与自适应连接规划（黄俊植）

该扩展把第 6.6 节的统计信息与代价模型用于访问路径和连接顺序选择，目标是在训练范围内降低估计误差、减少中间结果，在越出范围时安全回退。设计上借鉴 PRICE 的统计特征 [14] 和 CardOOD 对分布偏移的关注 [15]，但只实现标准库中的轻量模型，不进行跨数据库预训练。

图 10 是中文教学库上的三表连接演示：按 SQL 书写顺序执行时，先连接选课与课程，产生 10 000 行中间结果，连接候选比较 20 000 次；自适应规划先用 `p.department = 'CS'` 过滤学生表，再按“学生→选课→课程”的顺序连接，中间结果只有 100 行，候选比较降为 200 次，两种执行的完整结果（含重复行）一致。

8.3 扫描感知的预测缓冲（曾子恒）

预测策略的设计见第 5 节。它针对的是单连接串行执行中“大扫描与热点点读交替”造成的缓存污染，以及热点迁移后频率历史的适应性。与原论文 [16] 不同，本项目没有并发长扫描，也没有在真实云数据仓库负载上评测，因此实验收益不能视为该论文的复现结果。实验结果见第 10.6 节。

8.4 受预算约束的 AI 语义算子（路茗淳）

借鉴 iPDB [17]，本项目把大模型的判断做成有明确输入、输出和代价的关系算子，而不是让模型直接生成并执行 SQL：

- 先由确定性的 SQL 执行器物化一次 SELECT，再把用户选择的 TEXT 列交给语义管线。**SemanticFilter** 保留模型判为匹配的完整原行；**SemanticProject** 附加布尔判



图 10　同一 SQL 的两种连接执行策略对比

断与短标签；**SemanticAggregate** 对标签做确定性计数。NULL 不发送给模型。

- 同一次运行内先按完整文本去重，再按批调用 Chat Completions 接口；响应必须对每个输入 ID 恰好返回一个结果，缺失、重复、越界或类型错误都会被拒绝。精确缓存默认容量 512 条、有效期 300 秒，缓存键包含文本、条件、模型和服务配置的摘要，不保存明文凭证。
- 默认最多 64 行、每批 8 条、最多 8 批，SQL 与推理总预算 30 秒；推理期间不持有数据库锁，普通 SQL 可以继续执行。
- 未配置模型时提供“关键词：词一 | 词二”的本地确定性演示模式，并明确标注“未调用大模型”。

系统也已接入上海交通大学校园大模型服务（`deepseek-chat`）完成小样本真实验收，验收记录保存在 `docs/campus_api_smoke.json`。图 11 为本地演示模式下的语义筛选结果。

9　用户界面与使用说明

9.1　浏览器工作台

工作台（图 12）由 `server.py` 提供本机 HTTP 服务，前端为不依赖框架的 HTML / CSS / JavaScript。主要区域包括：

- 左侧：数据库对象列表，显示每张表的行数，点击字段可插入 SQL；
- 顶部：事务状态、缓冲区命中率、缓存页、实际读页与写页次数；
- 中部：带语法高亮和行号的 SQL 编辑器、示例下拉菜单，`Ctrl + Enter` 执行选区



图 11　语义筛选算子：输入 5 行课程，筛出与“数据库 / 查询 / 索引”相关的 1 行（本地规则演示）

或全部语句；

- 结果区：“查询结果”与“执行计划”两个标签页，显示行数和耗时；
- 工具区：AI 助手、扫描对比、一致性检查和研究实验（可信改写、优化模式、缓冲策略、语义算子）四个面板；
- 浅色 / 深色 / 跟随系统三种主题。

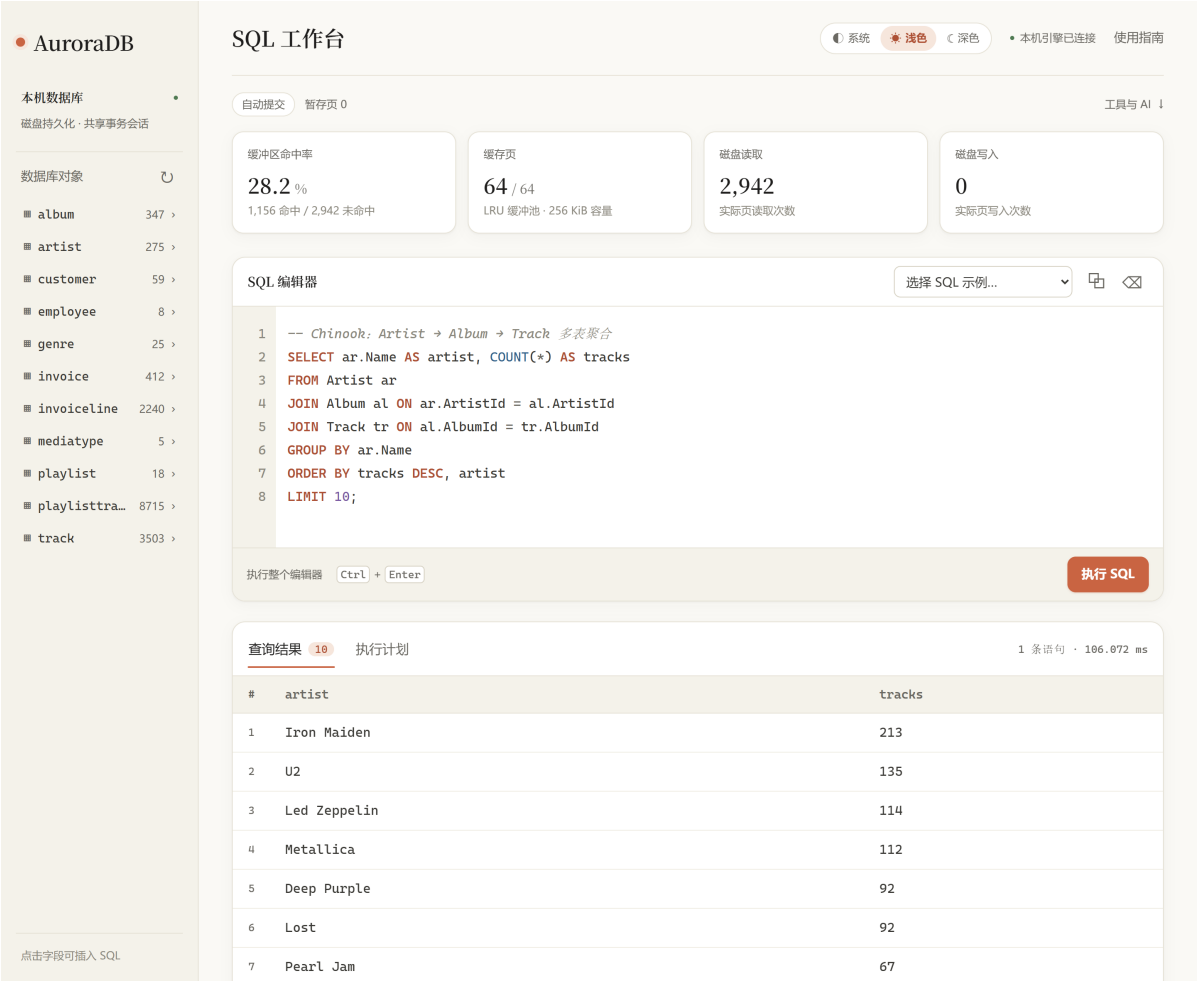


图 12　AuroraDB 浏览器工作台（Chinook 数据库，执行“艺术家与曲目 Top 10”示例）

9.2 命令行界面

不带 `--web` 参数启动时进入交互式 REPL，支持多行语句、分号结束、`SHOW TABLES` / `DESCRIBE` 等诊断命令；也可以用 `--execute` 执行一段 SQL，或用 `--file` 执行 UTF-8 编码的 SQL 脚本。

9.3 运行方式

Windows 可执行程序 双击仓库根目录下的启动脚本即可：*启动工作台*`.cmd` 打开中文教学库，*启动* `Chinook.cmd` 首次运行时导入 Chinook 数据集（独立目录，不影响教学库）。也可以在命令行中运行：

```
dist\AuroraDB.exe --web --demo --db data\aurora
dist\AuroraDB.exe --dataset chinook --web --port 0 --db data\chinook
```

源码运行（Windows / macOS / Linux，Python ≥ 3.12）

```
python -m auroradb --web --demo                  # web workbench + teaching data
python -m auroradb --dataset chinook --web --db data/chinook
python -m auroradb -e "SELECT * FROM students;"   # one-shot SQL
python -m auroradb                               # interactive REPL
python -m unittest discover -s tests             # run all tests
```

AI 功能 将 `ai.example.json` 复制为 `ai.local.json` 并填入服务地址与密钥后，工作台自动启用 AI 功能；该文件被 `.gitignore` 排除，打包脚本也会拒绝包含带密钥的配置文件。

9.4 安全设计

SQL 只被解析成数据结构，执行器按规则逐个计算，从不调用 `eval`；语法树深度受限；不支持的语法在出错位置明确报错。HTTP 服务只监听 127.0.0.1，并校验 Host 头（防止 DNS 重绑定）与 Origin 头（拒绝跨站请求）；写操作还需携带启动时随机生成的令牌（`X-Aurora-Token`，用常数时间比较）；请求体必须是不超过 512 KiB 的 JSON，研究实验另有执行预算。

10 测试与实验

10.1 测试方法

测试使用 Python 标准库 `unittest`，分为以下几类：

- **单元测试**：词法、语法、绑定、各算子、页编码、B+ 树分裂与范围扫描、WAL 帧解析等。

- **故障注入与恢复测试**：在提交流程的不同步骤模拟进程崩溃，检查重启后数据与日志的状态；构造损坏的页和日志，检查系统报错而不是静默接受。
- **差分测试**：以固定种子 20261009 随机生成 180 条 SELECT，分别在 AuroraDB 与 SQLite 上执行，按多重集合比较结果；连接扩展另有 120 条随机查询，在 heuristic 与 adaptive 两种模式下都与 SQLite 核对。
- **集成测试**：真实 HTTP 请求下的工作台接口、Host / Origin / 令牌校验、错误隔离与并发查询；可执行程序的冒烟测试。
- **研究扩展测试**：可信改写的正例与反例、学习模型的就绪与回退、预测缓冲的结果一致性、语义算子的协议校验与预算。

10.2 测试结果

全部 279 项测试通过（失败 0，错误 0），按子系统的分布见表 5。测试结果由 `tools/verify.py` 生成并写入 `docs/test_results.json`，测试总数不手工填写。

表 5 自动化测试分布（共 279 项，全部通过）

子系统	测试文件（测试数）	合计
存储与缓冲	storage (25)、integrity (20)、predictive_buffer (12)、integration_storage (10)	67
SQL 编译与执行	sql (23)、regressions (18)、engine (16)、diagnostics (6)	63
优化：改写 / 学习 / 连接	research_optimizer (29)、join_forms (15)、research_audit (12)、join_optimizer (6)、optimizer_demo (3)	65
AI 语义、服务与交付	ai (22)、semantic (17)、server (13)、ai_deadlines (9)、advanced_server (9)、ai_integration (8)、submission (5)、chinook_dataset (1)	84
合计		279

10.3 实验设置

所有实验脚本在临时数据库中运行，使用固定的数据生成方式与随机种子，把参数、时间、原始计数和源码 SHA-256 写入 JSON，源码变化后旧的实验证据会被判定为过期。“读页”指数据库文件的应用层页读取次数，操作系统文件缓存保留；耗时为包含 Python 开销的墙钟时间，只用于同一批次内的相对比较。

10.4 可信改写

实验在 5 000 行、16 页缓存的数据库上构造 13 个案例（`docs/optimizer_research.json`），全部符合预期，代表性案例见表 6。在 OR → IN 正例中，扫描行数由 5 000 降为 2，读

页由 109 降为 3，中位耗时由 70.05 ms 降为 1.90 ms。“短查询回退”案例虽为 proven，但原查询只需 0.04 ms，未达到 2 ms 门槛，因此不建议替换；未知外部候选即使结果一致也只得到 tested；插入数据后的反例、布尔与数值混淆、外连接与重复行变化都被判为 unknown 并拒绝。

表 6　可信改写实验的代表性案例

案例	验证级别	结果一致	建议替换
OR → IN 索引改写	proven	是	是
NULL 与重复 AND 删除	proven	是	是
排序同值组比较	proven	是	是
左连接可空列上的 OR	proven	是	否（不够快）
短查询回退	proven	是	否（原查询 $<$ 2 ms）
未知外部候选（仅当前数据一致）	tested	是	否
插入数据后的反例	unknown	否	否
布尔 / 数值混淆反例	unknown	否	否
外连接反例	unknown	否	否

10.5　学习型优化

36 条训练查询、9 条不参与训练的留出查询，每种模式每条执行 3 次（表 7）。平滑 Q-error 定义为 $\max\left(\frac{a+1}{e+1}, \frac{e+1}{a+1}\right)$，$a$、$e$ 分别为实际与估计行数。自适应模式下估计误差显著下降，访问路径改变 5 次，没有出现超过 15% 且 1 ms 的退化，全部结果一致；P95 耗时基本持平，说明更准确的估计并不必然带来所有查询的加速。阈值单调性检查 101 个点，违规 0 次。

表 7　学习型优化的留出评估（9 条查询）

指标	heuristic	adaptive
平滑 Q-error 中位数	1.850	1.027
平滑 Q-error P95	2.624	1.182
9 条查询中位耗时之和 (ms)	1342.9	847.6
逐查询中位耗时的中位数 (ms)	147.5	68.6
逐查询中位耗时的 P95 (ms)	253.1	255.8

连接扩展方面，三表演示用例的连接候选比较由 20 000 次降为 200 次（图 10）；

100×100 行的范围连接测试中，需要检查完整 ON 条件的候选由 10 000 次降为 4 950 次，结果一致。该计数不含排序与二分的内部比较，不能直接等同于耗时改善比例。

10.6 预测缓冲

6 000 行数据，四类负载（热点点读夹大扫描、重复范围查询、热点迁移、纯扫描），缓存 8 / 24 / 64 页，采样 1 / 4 / 8 / 32，每种配置 3 轮并随机化顺序，共 180 次运行、48 组对照，所有查询结果与基准一致。表 8 与图 13 给出采样数为 8 时与同版 LRU 的对比（Linux 环境，2026-10-28 加入扫描准入后的版本）。

表 8 预测策略与同版 LRU 的对比（采样 8；耗时比 = 预测 / LRU，小于 1 表示更快）

负载	缓存页	LRU 读页	预测读页	读页减少	耗时比
热点 + 大扫描	8	4011	3989	0.55%	0.962
	24	3769	3584	4.91%	0.747
	64	3769	3288	12.76%	0.765
重复范围	8	2496	2496	0.00%	0.995
	24	2496	2496	0.00%	0.978
	64	2496	2496	0.00%	0.985
热点迁移	8	7497	7481	0.21%	0.951
	24	7493	7136	4.76%	0.735
	64	7493	6804	9.20%	0.761
纯扫描	8	3696	3696	0.00%	1.129
	24	3696	3696	0.00%	1.114
	64	3696	3696	0.00%	1.140

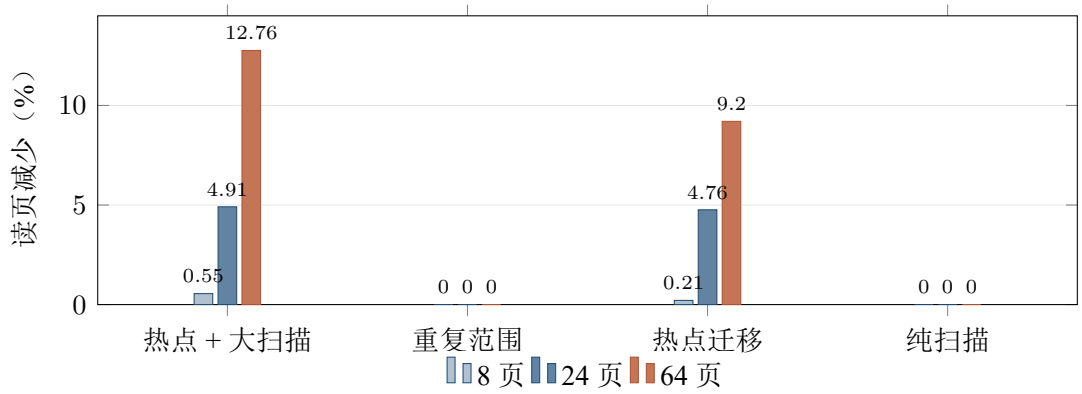


图 13 预测策略相对 LRU 的文件读页减少比例（采样 8）

在预先选定的“热点 + 大扫描、24 页、采样 8”配置上，预测策略的平均总耗时由 320.8 ms 降为 239.6 ms（降低 25.3%），文件读页减少 4.91%，独立的同场五轮复测降低 23.7%。耗时改善同时来自读页减少和避免不必要的页对象复制，不能全部归因于磁盘 I/O。重复范围查询没有读页收益，纯扫描负载下预测策略慢 11%–14%，因此它不是普遍更优的策略，默认仍使用 LRU。

10.7 语义算子的批处理与缓存

输入 64 行、完整文本去重后 32 条，使用确定性的本机模拟服务（每次请求人为等待 3 ms）对比三种方式（表 9）。调用次数的下降来自同次运行内去重、批处理和跨运行的精确缓存，三种方式的结果与预先定义的期望完全一致。由于使用的是模拟服务，这里的耗时不代表真实大模型的性能，也不评估模型判断的语义质量。

表 9 语义管线的调用次数与耗时（模拟服务）

方式	批大小	HTTP 调用	缓存命中	总耗时 (ms)
逐条调用	1	32	0	213.4
批量（冷缓存）	8	4	0	45.6
批量（热缓存）	8	0	32	1.4

10.8 公开数据集

Chinook [18] 是一个以音乐商店为背景的公开示例数据库（MIT 许可）。导入器 `tools/import_chinook.py` 把其中 AuroraDB 支持的列类型映射为 INTEGER、REAL、TEXT、BOOLEAN，跳过视图、触发器与二进制列，共导入 11 张表、15 607 行（Track 3 503、PlaylistTrack 8 715、InvoiceLine 2 240 等），每张表的预期行数由测试核对。Chinook 用于展示多表连接、聚合和索引计划；定量的性能结论仍以固定种子的受控实验为准。

10.9 有效性威胁

有限的 SQL 子集、单连接串行执行和小规模合成负载限制了结论的外部有效性；有限数据上的结果比较不能证明任意数据库上的等价；墙钟耗时受 CPU 调度和宿主机负载影响，跨批次的数字不能直接比较；文献中的性能结果只用于介绍方法，不作为本项目的结论。

11 小组分工与完成情况

小组共四人，按“一个人负责一个子系统和一个对应创新点”的方式分工；需求分析、总体架构讨论、接口联调、代码与结果核验、测试复现、报告撰写和答辩准备

由四人共同参与。模块负责人承担该部分的设计、实现组织与核验责任。经小组确认，四人工作量占比各为 25%。

表 10　小组分工与完成情况

成员	角色	负责内容	主要源文件 / 测试
钟国鹏（组长）	SQL 编译与可信查询执行	SQL 词法 / 语法分析、名称绑定、访问路径选择；扫描、连接、聚合与事务执行；创新点为可信查询改写（规则证据、结果核验、收益门槛与保守推荐）；工作台中 SQL 编辑器、执行计划、扫描对比与可信改写面板；小组协调与汇报整合	`sql.py` `engine.py` `optimizer.py`；test_sql、test_engine、test_regressions、test_diagnostics、test_research_optimizer
曾子恒	数据存储与预测缓冲	页格式与 CRC、堆表、B+ 树、WAL 与崩溃恢复、一致性检查；LRU 缓冲与扫描感知预测淘汰、扫描准入与页对象所有权优化；缓冲实验	`storage.py` `integrity.py`；test_storage、test_integration_storage、test_integrity、test_predictive_buffer
黄俊植	统计学习与性能优化	列统计、轻量基数 / 读成本 / 连接成本模型、有界工作负载反馈、3–6 表内连接动态规划、范围连接与连接规范化、置信度 / 域外 / 预算回退、执行计划诊断，以及训练 / 留出 / 漂移实验	`learning.py` `workload.py` `join_planning.py` `join_optimizer.py`；test_join_optimizer、test_join_forms、test_research_audit
路茗淳	AI 语义、交互与交付	语义筛选 / 分类 / 聚合算子、批处理与缓存协议、AI 接口与校园模型接入；浏览器工作台框架、命令行、HTTP 安全校验；打包、测试脚本与文档生成	`semantic.py` `ai.py` `server.py` `__main__.py` `web/`；test_semantic、test_ai*、test_server

12　局限与展望

- **SQL 覆盖**：没有子查询、窗口函数、RIGHT / FULL JOIN、外键、复合索引、ALTER TABLE 与视图。
- **并发**：单写者、单事务设计，没有 MVCC 与并发控制协议；预测缓冲的并发扫描

场景未被研究。

- **存储**：JSON 页体可读但空间利用率低；B+ 树删除采用延迟合并，删除大量数据后需要重建；不回收不可达的溢出页与目录分片。
- **可信改写**：目前只有 3 条规则且只改写 WHERE；证明只覆盖本项目实现的 SQL 方言。后续可增加谓词下推、常量折叠等规则，并对证明不了的候选引入 SMT 求解或自动反例搜索。
- **学习与 AI**：没有跨数据库预训练与在线漂移理论保证；语义算子的真实模型质量需要更大的人工标注集评估。

13 结论

本组从磁盘页、B+ 树和预写日志出发，自研实现了包含存储管理器、缓冲管理器、查询处理器、执行引擎和用户界面的关系数据库 AuroraDB，满足课程方案 1 的全部组件要求，并以 279 项自动化测试（含与 SQLite 的差分测试）验证其正确性。在可观察的内核之上，四位成员分别完成了可信查询改写、统计学习与自适应连接、预测式缓冲和 AI 语义算子四项扩展。这些扩展有一个共同的设计取向：**把“看起来更好”与“证明是对的”分开，证据不足时回到确定、保守的路径**。可信改写只替换经规则证明、全量核验且实测更快的查询；学习模型越界即回退；预测缓冲证据不足即回到 LRU；AI 算子的输出受协议校验与预算约束。我们认为，这种“可验证的智能优化”是把研究思想落到可运行系统时最重要的工程原则。

参考文献

[1] Codd E F. A relational model of data for large shared data banks[J]. Communications of the ACM, 1970, 13(6): 377–387.

[2] Silberschatz A, Korth H F, Sudarshan S. Database System Concepts[M]. 7th ed. New York: McGraw-Hill, 2019.

[3] Garcia-Molina H, Ullman J D, Widom J. Database Systems: The Complete Book[M]. 2nd ed. Upper Saddle River: Pearson Prentice Hall, 2008.

[4] Bayer R, McCreight E. Organization and maintenance of large ordered indexes[J]. Acta Informatica, 1972, 1(3): 173–189.

[5] Comer D. The ubiquitous B-tree[J]. ACM Computing Surveys, 1979, 11(2): 121–137.

[6] Selinger P G, Astrahan M M, Chamberlin D D, et al. Access path selection in a relational database management system[C]//Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data. 1979: 23–34.

[7] Graefe G. Query evaluation techniques for large databases[J]. ACM Computing Surveys, 1993, 25(2): 73–169.

[8] Mohan C, Haderle D, Lindsay B, et al. ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging[J]. ACM Transactions on Database Systems, 1992, 17(1): 94–162.

[9] O'Neil E J, O'Neil P E, Weikum G. The LRU-K page replacement algorithm for database disk buffering[C]//Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data. 1993: 297–306.

[10] Leis V, Gubichev A, Mirchev A, et al. How good are query optimizers, really?[J]. Proceedings of the VLDB Endowment, 2015, 9(3): 204–215.

[11] Chu S, Wang C, Weitz K, et al. Cosette: an automated prover for SQL[C]//Proceedings of the 8th Biennial Conference on Innovative Data Systems Research (CIDR). 2017.

[12] Narasayya V, Chaudhuri S. Leveraging query optimizers to verify the soundness of LLM-based query rewrites for real-world workloads, and more![C]//Proceedings of the Conference on Innovative Data Systems Research (CIDR). 2026. https://www.vldb.org/cidrdb/papers/2026/p33-narasayya.pdf.

[13] Fathollahzadeh S, Mansour E, Boehm M. ReSequel: robust LLM-assisted query rewriting and optimization using templatization and sampling[J]. Proceedings of the VLDB Endowment, 2026, 19(10): 2880–2893. DOI: 10.14778/3828612.3828639.

[14] Zeng T, Lan J, Ma J, et al. PRICE: a pretrained model for cross-database cardinality estimation[J]. Proceedings of the VLDB Endowment, 2024, 18(3): 637–650. DOI: 10.14778/3712221.3712231.

[15] Li R, Zhao K, Yu J X, Wang G. CardOOD: robust query-driven cardinality estimation under out-of-distribution[J]. The VLDB Journal, 2026, 35(4): 28. DOI: 10.1007/s00778-026-00979-3.

[16] Khalaji M, Vanderkooy T, Guo R B, Daudjee K. Sampling-based predictive database buffer management[J]. Proceedings of the VLDB Endowment, 2025, 18(13): 5569–5581. DOI: 10.14778/3773731.3773734.

[17] Kumarasinghe U, Liu T, Mahmood A R, Liu C, Aref W G. iPDB: optimizing semantic SQL queries[EB/OL]. arXiv: 2601.16432, 2026. https://arxiv.org/abs/2601.16432.

[18] Rocha L. Chinook Database, version 1.4.5[EB/OL]. https://github.com/lerocha/chinook-database.

A 复现步骤与仓库结构

在仓库根目录依次执行以下命令，可重新运行全部测试与实验并生成证据文件：

```
python tools/verify.py                  # 279 tests -> docs/test_results.json
python tools/optimizer_experiment.py    # -> docs/optimizer_research.json
python tools/buffer_experiment.py       # -> docs/buffer_research.json
python tools/semantic_experiment.py     # -> docs/semantic_research.json
python tools/join_optimizer_experiment.py
python tools/build_exe.py               # build dist/AuroraDB.exe
python tools/executable_smoke.py        # smoke-test the executable
```

表 11 仓库目录结构

路径	内容
auroradb/	数据库内核、研究扩展、HTTP 服务与前端（web/）
tests/	279 项自动化测试
tools/	实验、构建、打包与文档生成脚本
dist/AuroraDB.exe	Windows 单文件可执行程序
docs/	本报告、设计与测试报告、软件工程文档、项目论文、使用说明，以及实验证据 JSON